arXiv:2003.03471v1 [cs.SE] 6 Mar 2020

SpellBound: Defending Against Package Typosquatting

Matthew Taylor[1], Ruturaj K. Vaidya[1], Drew Davidson[1], Lorenzo De Carli[2], and Vaibhav Rastogi[3]

[1]University of Kansas
[2]Worcester Polytechnic Institute
[3]Google

Abstract

Package managers for software repositories based on a single programming language are very common. Examples include npm (JavaScript), and PyPI (Python). These tools encourage code reuse, making it trivial for developers to import external packages. Unfortunately, repositories' size and the ease with which packages can be published facilitates the practice of *typosquatting*: the uploading of a package with name similar to that of a highly popular package, typically with the aim of capturing some of the popular package's installs. Typosquatting has serious negative implications, resulting in developers importing malicious packages, or—as we show—code clones which do not incorporate recent security updates.

In order to tackle this problem, we present SpellBound, a tool for identifying and reporting potentially erroneous imports to developers. SpellBound implements a novel typosquatting detection technique, based on an in-depth analysis of npm and PyPI. Our technique leverages a model of lexical similarity between names, and further incorporates the notion of package popularity. This approach flags cases where unknown/scarcely used packages would be installed in place of popular ones with similar names, before installation occurs. We evaluated SpellBound on both npm and PyPI, with encouraging results: SpellBound flags typosquatting cases while generating limited warnings (0.5% of total package installs), and low overhead (only 2.5% of package install time). Furthermore, SpellBound allowed us to confirm known cases of typosquatting and discover one high-profile, unknown case of typosquatting that resulted in a package takedown by the npm security team.

1 Introduction

Package managers are tools which automate the complex task of deploying 3rd-party dependencies into a codebase, abstracting away the provenance of the dependency; when the user invokes a command to install the package by name, the given package will be downloaded from a remote repository, alongside the full set of additional packages upon which it transitively depends. One of the most common uses of package managers is in the context of large repositories of code packages based on a single programming language. Package managers are undeniably useful, with open, free-for-all repositories like npm for Node.js/JavaScript, PyPI for Python, the NuGet Gallery for Microsoft's .NET Framework, and crates.io for Rust, collectively serving billions of packages per week. Despite their utility, package managers also come with problems.

The ease with which code can be imported facilitates incorrect imports. Installing an unintended code dependency can be catastrophic, but happens as easily as mistyping a single character on the command line. Furthermore, the open, uncurated nature of these repositories means that any developer can upload a package with a name of their choosing and it will be treated with equal trust as any other package in the repository. This circumstance gives rise to *typosquatting*, whereby a developer uploads a "perpetrator" package that is *confusable* with an existing "target" package due to name similarity.

The process by which typosquatting acts is simple: the user, intending to install the target package, accidentally requests the name of the confusable perpetrator package. Determining why perpetrators packages are created and uploaded is a challenging and ill-defined problem, as solving it requires inferring the intent of the package author. The perpetrator may wish to intentionally confuse users into installing a malicious payload, seek to increase the visibility of their own benign code, or may have created a confusable name by happenstance, without realizing it. A typosquatting perpetrator might even upload a placeholder package to prevent an attacker from leveraging the given name. Regardless of the intent, the result is the same: users are confused into importing the incorrect package into their code.

Typosquatting has numerous detriments, both to developers who integrate a perpetrator package into their codebase, and to the end-users of such a codebase. An overtly malicious perpetrator may include Trojan functionality that attacks the

client when run [10, 12]. Additionally, many package managers invoke configuration hooks bundled with the package at install time, often manifested as shell scripts that run with the privileges of the user. Multiple packages that open reverse shells when installed have been removed from npm [9, 11, 13]. Even in cases where the perpetrator package is not overtly malicious, it can confuse the user and weaken the integrity of the system. Ironically, a perpetrator might clone a victim to keep it out of the hands of an attacker but allow the clone to fall behind as the target is updated, exposing users of the clone to latent vulnerabilities that have been patched out of the target.

Typosquatting is a difficult problem to detect manually, as it confuses manual inspection by definition. In this work, we develop SpellBound, a novel typosquatting detection technique to discover and prevent incidents of typosquatting before they can damage the user. SpellBound can be used to detect typosquatting incidents before they happen, or to detect possible perpetrator packages within a package repository.

To illustrate typosquatting, and the benefits of our approach, consider the example of *loadsh*, an npm package that SpellBound reported to be typosquatting the popular *lodash* package. Because *loadsh* is a transposition of the "a" and "d" characters of *lodash*, our techniques detected that the package names are easily confusable. We confirmed that *loadsh* was being used uninentitionally by emailing the maintainers of packages that used *loadsh*. Three *loadsh*-dependant package maintainers responded to our email, all of whom acknowledged that they had intended to install *lodash* and indicated that they would change their dependency. Many of the packages using *loadsh*, including those maintained by our respondents, had been victims for over a year.

The *loadsh* incident exemplifies several stealthy aspects of package name typosquatting; not only are the developers who use *loadsh* victims of a typosquatting attack, so are packages that *transitively* depends on *loadsh* (i.e. those codebases that depend on a package that accidentally uses *loadsh*). Thus, it is possible to be a victim to a typosquatting attack without personally making a typo. Another difficulty in detecting packages like *loadsh* is that they may not exhibit malicious behavior. Indeed, *loadsh* does not include any malicious functionality - the perpetrator package is an exact snapshot copy of *lodash* version 4.17.11, the current version of the target at the time at which the typosquatting package was created (the target package *lodash* is currently at version 4.17.15). Nevertheless, the perpetrator still has a negative impact; because the perpetrator package has not been updated, its victims were effectively using an outdated version of *lodash*. In the case of this example, the older version has been reported to contain prototype pollution vulnerabilities [15], effectively leaving victims of *loadsh* open to attacks that have already been patched in the current version of *lodash*. When *loadsh* was reported, 63 other package depended on it. Each of these dependents were, by extension, vulnerable to prototype pollution. After SpellBound reported *loadsh* as a typosquatting perpetrator, we contacted the npm security team, who verified our results, deprecated *loadsh*, and took over ownership of the package.

As indicated by the *loadsh* example above SpellBound can be used to detect if a given package is a perpetrator of a typosquatting attack. At high level, SpellBound intercepts and analyzes package install requests. First, it checks if a given package is not popular. If so, it checks if the given package's name is *lexically similar* to that of a popular one (we describe and motivate our notions of popularity and similarity in Section 3). If both conditions are met, SpellBound concludes that the user is at risk of installing a typosquatting perpetrator, and issues an alert before the package is fetched. Furthermore, it presents a suggestion for the likely correct package name that is being typosquatted.

Overall, our work makes the following contributions:

- We highlight the security implications of typosquatting.
- We study the extent to which typosquatting exists in npm and PyPI.
- We present SpellBound, an enhancement to the package manager front-end which protects users against typosquatting attacks.
- We evaluate the efficacy of SpellBound. We show it offers a higher level of security while incurring a 2.5% overhead during package installation. Additionally, we demonstrate that SpellBound is non-intrusive, as it affects less than 1% of all weekly downloads for popular package repositories.

The rest of this paper is structured as follows: Section 2 provides background on package managers and typosquatting attacks. Section 3 describes and motivates the design of SpellBound. Section 4 evaluates SpellBound's performance. Section 5 discusses limitations and possible extensions of our work. Section 6 examines the related work. Finally, Section 7 concludes the paper.

2 Background

In this section, we give background information necessary to understand the need for a tool like SpellBound. In particular, we show how the current landscape of package management enables typosquatting, and describe previous attacks that use typosquatting to deliver malicious payloads. We discuss the context that makes typosquatting a pernicious problem for many of the repository stakeholders, including end-users of applications, application developers, package providers, and the maintainers of repositories themselves.

	npm	PyPI
Packages	1,221,705	221,041
Weekly Downloads	17,872,179,641	997,624,343
Avg. Dependency Tree Size	57.27	4.58

Table 1: Usage statistics for npm and PyPI: Both repositories serve significant numbers of highly interdependent packages on a weekly basis.

2.1 Package Repositories

The use of package repositories for managing dependencies is incredibly popular. They simplify the use of third-party code, which in turn has obvious benefits. It encourages code reuse; it allows expertly-written and well-vetted codebases to be deployed by more developers; and it leverages the knowledge of the broader software development community even for highly-custom projects. For these reasons, successful repositories may grow to enormous size. The first two rows of Table 1 show the current size and weekly download counts for npm and PyPI, as reported by the repository maintainers. As the table shows, they contain hundreds of thousands (in the case of PyPI), or even millions (in the case of npm) of publicly available packages. The total number of weekly downloads served are nearly 1 billion in the case of PyPI and over 17 billion in the case of npm.

Much of the complexity of package management is due to the interdependence of packages. For example, the popular npm package *webpack-dev-server* (6.6 million weekly downloads) declares 33 dependencies of its own. These 33 dependencies require further packages to be installed (the transitive dependencies of *webpack-dev-server*). In total, *webpack-dev-server* has 391 transitive dependencies. Running *webpack-dev-server* requires that all 391 packages are installed. Furthermore, these packages span many distinct development teams, each of which may update out of step with one another, introducing new functionality and behavior. This is in line with the general trend of code reuse in software development: a recent report by the software security company Contrast Security found that 79% of application code came from third parties [34]. Given the bulk of code existing in dependencies, it is infeasible to expect developers to manually vet every package or piece of code that they integrate into their project.

Package manager frontends automate the complex and tedious task of fetching, configuring, and updating a package and its transitive dependencies. When a user issues a command like `npm install webpack-dev-server`, the frontend relies on the package's metadata to build a spanning tree of the package dependency graph (referred to internally as the package dependency tree), and then installs each package node in the tree. Similarly, the command `npm update` updates the package dependency tree for the current set of packages, and ensures that the most recent compatible versions of dependencies are deployed. The third row of Table 1 shows the average size of the dependency tree for the two package managers we study. It is notable that there is significant interdependence among packages.

While package managers save users a significant amount of time, they do not help with the herculean task of vetting imported code; if anything, they complicate it. The key design goal of package manager frontends is that they make fulfilling dependencies opaque to the user. As a result, the provenance of a package is also obscured - a user need not explicitly trust the developer of a package they (transitively) use, nor even know *who* uploaded the code to the package repository. Once a package is registered to the repository, it is given equal trust as any other package on the repository, and may be freely integrated into applications or other packages.

Characterization of Package Downloads: The majority of package downloads are due to a small number of packages. Based on the self-reported repository download counts, we classified the popularity of packages across npm and PyPI. Figure 1 and Figure 2 show the distribution of downloads across npm and PyPI, respectively. A majority of the packages for both repositories are downloaded between zero and ten times per week. Only a small fraction of packages see a high degree of popularity. However, the packages composing the smallest portion of each figure actually receive more downloads than the packages in all remaining portions combined. Locating desired packages in this ocean of unpopular ones without assistance can be challenging.

2.2 Factors Contributing to Typosquatting

The automated nature of package managers has enormous utility. However, this also enables misuse, namely typosquatting. We propose that the following aspects of package repositories contribute to the threat of typosquatting:

- The open-source nature of repositories means that any user can upload a package, and it will be given equal trust with any other package.
- The provenance of a package is opaque to the user, and the interdependence between packages makes their behavior difficult to vet manually.
- The distribution of packages means there are a small number of "juicy" typosquatting targets, and a large number of package from which a typosquatting attack could be launched.

We now review select cases of historical typosquatting, and describe the challenges in detecting typosquatting reliably.

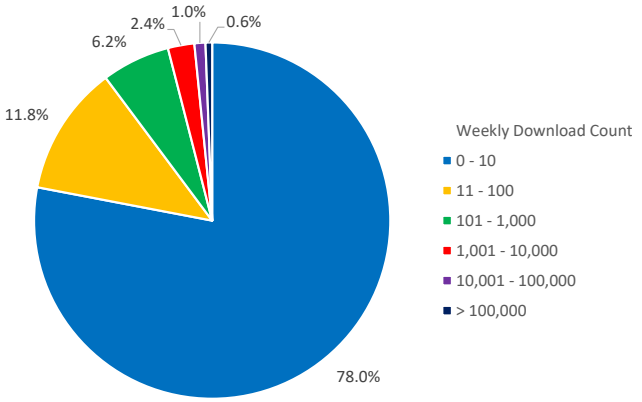

Figure 1: Download distribution for packages on npm.

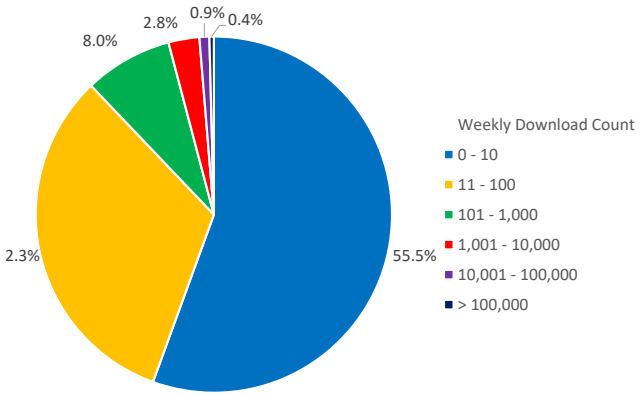

Figure 2: Download distribution for packages on PyPI.

2.3 Historical Package Typosquatting

The degree to which typosquatting has historically occurred is difficult to capture, due in part to the highly subjective nature of what constitutes typosquatting. Indeed, there exist cases of package name similarity where intent may appear benign or ambiguous. In practice, most packages that are flagged by repositories exhibit overtly malicious functionality, and are retroactively deemed typosquatting perpetrators by a qualitative manual analysis. It is also important to observe that not all malicious packages perform typosquatting.

As an example of the complexities of determining typosquatting and its intent, consider the *js-sha3* typosquatting campaign. On October 25th, 2019, 25 packages were simultaneously identified by Microsoft Vulnerability Research and taken down by the npm security team: *zs-sha3, ns-sha3, ks-sha3, jw-sha3, jsmsha3, js-wha3, js-sxa3, js-sla3, js-sja3, js-sia3, js-shq3, js-she3, js-shc3, js-shas, js-sha7, js-rha3, js-qha3, js-cha3, js-3ha3, jr-sha3, jq-sha3, jc-sha3, j3-sha3, hs-sha3, bs-sha3*.

Upon close inspection, all those packages were determined to have malicious intent, and all package names were close, according to Levenshtein distance, to the victim package *js-sha3*. However, not all packages names were likely to confuse the user. For example, *js-sxa3* requires replacing the "h" with an "x". It is unlikely that a developer would misremember *js-sha3* as *js-sxa3* (the package being an implementation of the SHA-3 algorithm). A typo is equally unlikely on a QWERTY keyboard, given the distance between "h" and "x". As discussed in Section 3.3, we take the stance of only flagging cases where there is strong likelihood that name similarity may confuse the user. While this causes us to ignore some cases (as *js-sxa3* above), it has the advantage to avoid generating an excessive number of warnings.

One may also be tempted to solve these ambiguities by always attempting to identify malicious intent, regardless of whether typosquatting occurs. In practice, this is challenging and currently impossible to achieve reliably. Source code analyses might be employed to catch overtly malicious behavior in a perpetrator package, but they have difficulty detecting obfuscated payloads. JavaScript is a particularly difficult target to analyze - recent work has shown that JavaScript can be automatically obfuscated to appear syntactically indistinguishable from benign code to modern detectors [25]. Furthermore, the highly dynamic nature of JavaScript means that malicious functionality may not appear until the script is deployed.

Currently, the standard technique for removing these packages is manual and reactive. Users who believe a package is performing malicious typosquatting can file a report to the repository maintainers, who will then investigate the claim. Should the maintainers agree with the reporter, the package will be removed. This approach does little to prevent the installation of malicious packages and fails to protect users from the consequences.

Despite the shortcomings of this approach, hundreds of package takedowns have been issued that involve package names similar to a popular target. We believe this number to be a lower bound on the total number of typosquatting attempts. Due to the ease with which packages can be registered to a repository, the differential of effort favors the attacker; the 25 packages reported by Microsoft above exhibit many of the hallmarks of automatic creation (which may contribute to the poor confusability of some of the entries), such as identical payloads. Thus, an attacker can outpace the current manual detection techniques through clever scripting. Many of the reported incidents of typosquatting with a malicious payload were active for months or even years before they were reported.

2.4 Consequences of Typosquatting

A package repository ecosystem consists of several distinct stakeholders, many of whom are adversely affected by typosquatting. We note some of the ways in which the consequences of confusing a perpetrator package for a victim package can be felt by these parties:

Attacks against end-users: The most subtle attack that uses typosquatting is when an adversarial uploader delivers a malicious payload as part of the dependency code, which is subsequently used as part of a user-facing application. This attack impacts the end-user of the application. Two highly-publicized incidents of this consequence involved a malicious payload that exfiltrated sensitive information such as credit card numbers [12] or cryptocurrency [13]. A stealthy adversary may attempt to obscure the payload by cloning the target package and adding the malicious functionality as a Trojan.

Attacks against developers using a package: An adversary may also target the developer who mistakenly requests the perpetrator package at install time. Both npm and PyPI allow packages to invoke shell scripts in order to configure and deploy the script, which run under the privileges of the invoking user. Since packages can be installed system-wide, the user may be the administrator, opening a vector for an adversary to do catastrophic harm to the developer's machine. A common choice for malicious package creators is to open a reverse shell, giving them full control of the victim's machine [14].

Degradation of functionality: Even when perpetrator does not deploy malicious code, they may still hinder operations. If the confusion is purely accidental, it is likely to be noticed well before the victim application is deployed. Nevertheless, this incidental confusion will at least waste time, the victim's time, in diagnosing the problem.

Latent vulnerabilities: If a perpetrator package is not detected immediately upon installation, it may remain latent in the victim's codebase for a significant period of time. A frequent cause of this is when a developer typosquats a target with a payload that is a clone of the current version of the package. While the victim experiences no initial consequences from using the wrong package, they are at the mercy of the perpetrator that the code will be kept in lockstep with the target. As in the case of *loadsh*, mentioned in Section 1, the clone may never be updated, meaning that the perpetrator is exposed to latent bugs and vulnerabilities that have been patched in the target [15].

Misattribution: Even if a perpetrator package replicates all of the target functionality, it nevertheless fragments the popularity of the target package. Thus, one minor consequence of typosquatting is that the target will not get as much credit as they would without the perpetrator. Misattribution can be found in packages like *asimplemde* on npm. In addition to typosquatting, this package contains identical functionality to *simplemde*. References attributing credit to the original author are the sole omissions from the duplicate package.

3 Detecting Typosquatting

Motivated by the number of historical instances, the ease of execution, and the severity of the possible consequences, we created SpellBound, a tool to detect typosquatting in package repositories. At a high level, SpellBound compares a given package name to a list of popular package names. If the given package name matches at least one of the popular packages after a set of allowed transformations (or signals), then it is considered to be a typosquatting suspect. In that case, SpellBound raises a alert and indicates the likely package being typosquatted before prompting the user to proceed.

3.1 SpellBound Workflow

The primary way in which we expect SpellBound to be deployed is as a user-facing utility that integrates with the package manager frontend and introspects upon packages before they are installed.

Figure 4 depicts the overall workflow of SpellBound, including both typosquatting detection and steps performed in the normal course of package installation. Algorithm 1 presents a description of typosquatting detection (steps 4 through 7 in the figure) in pseudocode.

The user initiates the process by triggering a package's installation from the command line, e.g., `npm install loadsh` (**step 1**). The package manager computes the dependency tree of the package, i.e. its transitive closure on the package graph (**step 2**). Subsequently, it discards all packages that are already installed, and thus do not need to be downloaded (**step 3**). At this point, the workflow triggers SpellBound's logic.

First, SpellBound considers each package queued to be installed (**steps 4-5**, lines 1-3 in Algorithm 1). A package is considered suspicious if its *popularity score* (explained in Section 3.4) is below a tunable threshold T_p, and there exists a popular (popularity $\geq T_p$) package with a similar name (similarity is discussed in Section 3.3). If this is the case, SpellBound flags the package and prompts the user (**step 5-6**, lines 4-8). The prompt displays a brief explanation of the warning, which includes both the name of the offending package, and the name of the package that most likely should be installed instead (an example prompt is shown in Figure 3). If the user decides to ignore the warning, the package is installed (**step 8**, line 6), otherwise the process is terminated. Note that *AbortInstallation()* in line 8 terminates the process for all queued packages, not just the one which was the object of the warning. In lines 9-10, any package which does not raise suspicion is directly installed without prompting the user.

3.2 SpellBound Batch Analysis

While we anticipate the workflow in Figure 4 to be the most common application of SpellBound, we also envision repository maintainers may want to periodically apply the same analysis in batch fashion to the *entire package repository*. This would simplify the task of identifying highly suspicious

```
Security warning: one of the packages about to be installed
may not be what you think. You are about to install loadsh
(2,593 downloads/week). If you intended to install lodash
(27,687,524 downloads/week) instead, enter "N" to abort the
installation. Continue? [Y/N]
```

Figure 3: Package installation prompt

packages. Our current implementation also supports this approach. In this mode, SpellBound receives as input the list of all package names. It then returns a list of candidate perpetrators, ranked by decreasing download count. Indeed, the *loadsh* package discussed in Section 1 was identified in this way; SpellBound's batch analysis ranked it as the seventh most popular typosquatting candidate matching a specific signal discussed in the next subsection.

3.3 Typosquatting Signals

SpellBound relies on the ability to identify pairs of packages with similar names; however, precisely defining the notion of similarity is challenging. Initially we experimented with thresholds on basic Levenshtein distance. However, we found this approach overly simplistic, and generating an enormous number of matches. These similarities are bound to happen purely due to the size of the repository: there are 9,371 3-letter packages in npm, and only 17,576 combinations of three lowercase English letters[1].

After extensively exploring alternative approaches, we designed a notion of similarity that relies on the disjunction of six possible signals (i.e., triggering one signal causes a pair of names to be considered similar). These signals were created by examining past typosquatting attacks and extending signals used to detect domain name typosquatting such that they apply to package repositories [29, 40]. The signals, along with descriptions and examples of genuine perpetrator/victim package pairs are listed below. Note that a majority of the examples used are historical typosquatting instances and have been removed, though all examples would be detected by SpellBound.

1. Repeated characters — the presence of consecutive duplicates in a package name. For example, *reequest* is typosquatting *request*.

2. Omitted characters — a restricted form of edit distance, not allowing arbitrary character substitutions and additions. The maximum allowed number of omissions is set to one. For example, *comander* is typosquatting *commander* and *require-port* is typosquatting *requires-port*.

3. Swapped characters — two consecutive characters have been swapped. For example, *axois* is typosquatting *axios*.

[1] Names can use other symbols, however most short names do not include them.

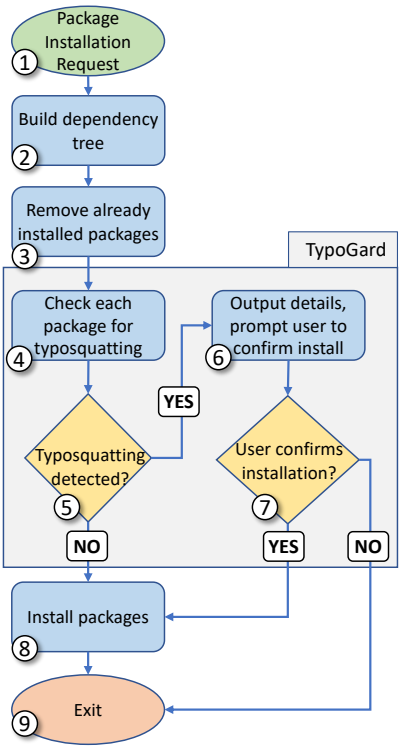

Figure 4: Modified package installation process with integrated typosquatting protection.

4. Swapped words — this signal depends on the presence of delimiters in a package name, where a delimiter is a period, hyphen, or underscore. This signal checks for any other ordering of delimiter-separated tokens in the package repository namespace. This signal checks for reordering with other delimiters as well. For example, *import-mysql* is typosquatting *mysql-import*.

5. Common typos — character substitutions based on physical locality on the QWERTY keyboard layout. This signal also checks for substitutions of characters with visual similarity. For example, *signqle* is typosquatting *signale*, *1odash* (with the number one) is typosquatting *lodash* (with the letter L), and *uglify.js* is typosquatting *uglify-js*. The rationale for checking for characters with visual similarity is that, even if users are unlikely to make the typo, they may overlook such packages if they are imported indirectly as malicious dependencies. These packages are not explicitly requested by the user, however, they can be seen during the installation process. Attackers could utilize this style of substitution in hopes that it could be confused with another at a glance.

6. Version numbers — the presence of integers located at the end of package names. Optional delimiters between the package name and the version number are also considered. For example, *underscore.string-2* is typosquat-

ting *underscore.string*. Note that *underscore.string-2* was previously undiscovered and SpellBound led us to find a latent vulnerability.

3.4 Package Popularity

Once the typosquatting detection scheme had been created, we required some formal definition of popularity to successfully implement SpellBound. This requirements stems from a fundamental belief that we posit, which is that only unpopular packages can be typosquatting perpetrators and only popular packages can be typosquatting targets. Popular packages are, by our definition, incapable of perpetrating typosquatting attacks. Next, we believe that there exists no incentive for an adversary to typosquat a package which receives an insignificant amount of attention. If a negligible number of users download that package, then an even smaller number of people could potentially misspell the name of that package and fall victim to the attack. By this token, a package which is downloaded thousands, millions, or even tens of millions of times per week, is a far more rewarding target.

The two main possibilities for quantifying package popularity were the number of downloads and the number of dependents. We decided to focus on the number of downloads because we believe it is a more indicative measure of true package usage. The public number of dependents counts only the number of other packages that have been uploaded to the repository that directly depend on a given package. Download count, on the other hand, counts the number of users who have downloaded that package either directly or indirectly through some arbitrarily long chain of dependencies.

Popularity based on download count requires the definition of a threshold to distinguish between popular and unpopular packages. This threshold is of crucial importance because the number of packages considered to be typosquatting depends directly on the number of packages considered to be popular. An exceedingly low threshold results in many typosquatting packages being considered popular, thus making their detection impossible. Conversely, an exceedingly high threshold may miss packages with are frequently downloaded and are victims of typosquatting. We use a data-driven approach, discussed in Section 4, to determine the threshold.

4 Analysis and Evaluation

In this section, we perform an in-depth analysis of SpellBound's tunable parameter, the popularity threshold, and we evaluate SpellBound's effectiveness in flagging suspicious package installs. Our goal is to answer the following questions:

1. Is it possible to determine an optimal popularity threshold based on repository characteristics? What is the impact of varying this threshold? (Section 4.2).

Algorithm 1 SpellBound typosquatting detection

Input: List I of packages to be installed
Input: Package graph G
Input: Popularity threshold T_P

```
for each p ∈ I do
    if Popularity(p) < T_P then
        if ∃p′ s.t. Popularity(p′) ≥ T_P and Similar(p, p′) then
            R ← UserConfirm?(p, p′);
            if R = True then
                Install(p);
            else
                AbortInstallation();
    else
        Install(p);
```

2. What is the effectiveness of SpellBound's typosquatting signals in identifying suspicious packages? (Section 4.3).

3. Is the latency introduced by SpellBound to the package installation process acceptable? (Section 4.3).

4.1 Dataset

In order to perform our analysis, we consider the entire package graphs for npm and PyPI. In particular, our analysis is based on snapshots of npm and PyPI which reflect their state on February 19, 2020. A high-level quantitative summary of both repository snapshots is given in Table 1.

4.2 Popularity Threshold

Download counts bear an obvious relationship to the popularity of a given package within the developer community. Precisely understanding this relationship however requires careful analysis of a software ecosystem. This is due to the fact that download counts on npm and PyPI represent more than the number of people who have installed a package. Packages are regularly downloaded by repository mirrors and bots which download all packages for analysis. These downloads are also recorded in a package's total download count. Based on estimates made by the creators of npm, a package can be downloaded up to 50 times per day without ever being installed by an actual developer [2].

Based on this estimate, we use 350 weekly downloads as an absolute lower bound for package popularity as packages with fewer than this number of downloads may have never been downloaded by an actual user. As seen in Figures 1 and 2, a majority of packages in both npm and PyPI receive fewer than 350 weekly downloads. The stipulation that a package must have, at the very minimum, 350 weekly downloads to be considered popular removes about 93.9% of npm packages

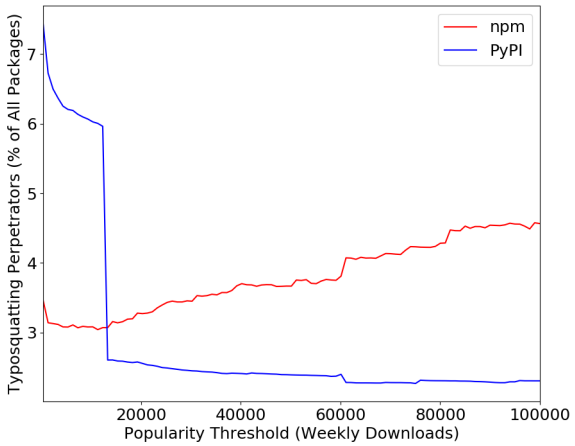

Figure 5: Relationship between popularity threshold and percent of repository typosquatting.

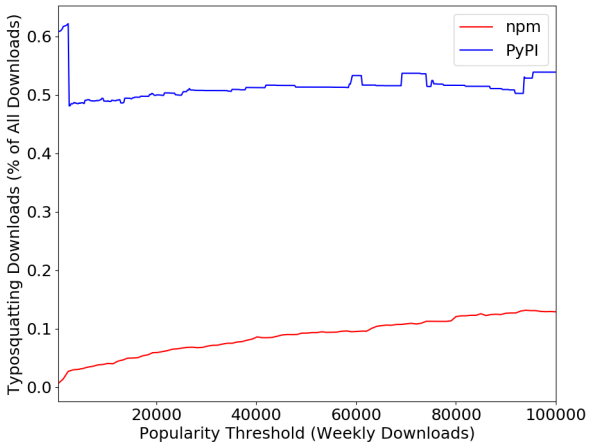

Figure 6: Relationship between popularity threshold and percent of weekly downloads containing a typosquatting package.

and about 93.3% of PyPI packages from consideration. Interestingly, this suggest that only a tiny fraction of packages in these repositories receive any meaningful attention and usage from the community. Due to the size of these repositories, however, this fraction still amounts to millions of downloads. As an upper bound, we consider packages with more than 100,000 downloads per week to be unquestionably popular. Packages above this upper bound make up the top 0.6% of npm and the top 0.4% of PyPI. The analyses we describe in this section aim at finding an appropriate threshold to separate popular packages from unpopular packages between these two bounds.

Effect of threshold on number of perpetrators: The first analysis aims to determine how the number of typosquatting targets influences the number of typosquatting perpetrators. This is a transitive test, which means a package is considered to be a typosquatting perpetrator if it, or any package in its dependency tree, fits our definition of typosquatting. Doing this emulates real-world conditions, as users typically would not install a package without installing its dependencies. The results of this analysis is depicted in Figure 5. Interestingly, the curves corresponding to npm and PyPI are fundamentally different. As the popularity threshold increases, the number of popular packages decreases. With this decrease in typosquatting targets, one would initially expect the number of typosquatting perpetrators to decrease. The trend for PyPI is consistent with this behavior. The sharp drop in perpetrators is due to a large number of packages that fit our definition of typosquatting that also have just over 13,000 weekly downloads. As soon as the popularity threshold crosses 13,000, these packages are considered to be popular and are therefore exempt from being typosquatting perpetrators, causing the drop in perpetrators.

In stark contrast contrast, npm's trend steadily increases. The number of typosquatting perpetrators grows in spite of the fact that the number of targets shrinks. This highlights an interesting phenomenon present in npm: there's a significant amount of package name similarity between reasonably popular packages. This idea is best exemplified by cases like those shown in Table 2. All of these packages have significant download counts. Examples like those found in Table 2 cause the unintuitive increase in perpetrators seen in Figure 5. For small popularity thresholds, both packages in these pairs are considered popular. However, as the threshold grows, it passes the weekly download count of the less popular package, which in turn turns the less popular package into a perpetrator. Ultimately, this process increases the number of perpetrators as the number of targets decreases.

Based on the analysis discussed above, we have chosen to select a popularity threshold of 15,000 weekly downloads. A popularity threshold of 15,000 weekly downloads is the lowest threshold which keeps the number of typosquatting packages reasonably low for both repositories. For both npm and PyPI, approximately 3% of all packages on each repository are potentially typosquatting for this threshold.

Effect of threshold on frequency of warnings: The second analysis examines how frequently packages that could be considered typosquatting are downloaded. It is important to understand this datum in order to get a sense of how frequently SpellBound will intervene during the package installation process. Maintaining the frequency of interventions low is important for two reasons. First, frequently interrupting a developer's workflow with warning notifications risks incurring in the well-known phenomenon of warning fatigue [19]. Second, it is reasonable to expect that the number of packages imported by mistake is a relatively small fraction of the overall number of packages imported by a developer. Therefore, a

Package Name	Weekly Downloads
object-assign	17,249,391
object.assign	10,843,774
isarray	30,271,796
is-array	69,131
is-buffer	19,143,770
isbuffer	35,684
memorystream	1,125,398
memory-stream	6,047

Table 2: Typosquatting cases with popular perpetrators.

	npm	PyPI
Repeated Characters	443	40
Omitted Characters	3827	412
Swapped Characters	514	63
Swapped Words	1732	77
Common Typos	4409	533
Version Numbers	1148	116
Total	**12073**	**1241**

Table 3: Number of packages triggering each typosquatting signal.

very high number of warning is likely to consist overwhelming of false positives [17].

This analysis, like the first, is transitive in order to emulate real-world conditions. Ideally, the number of alerts asking the user if they are sure they would like to install the requested package should be kept close to zero. The results of this analysis are show in Figure 6. In this test, trends for both repositories are noticeably similar than the trends in Figure 5. According to this figure, with any reasonable popularity threshold, the percentage of weekly downloads which result in a warning from SpellBound is around 0.1% for npm and around 0.5% for PyPI. In other words, SpellBound generates on average a warning every 200 to 1000 package installs, which we consider an acceptable burden for a developer.

4.3 Signal Detection Rates

In this section, we consider the effectiveness of the typosquatting signals used to determine package name similarity (ref. Section 3). The signals we chose to include in this implementation of SpellBound detected approximately 60% of known past attacks reported by the npm security team as typosquatting. While this number may appear low, it chiefly stems from qualitatively different definitions of typosquatting used by npm and us.

For example, npm considers *ruffer-xor*, *bwffer-xor*, *bufner-xor*, and similar ones to be typosquatters of *buffer-xor*. While the former names are all at a Levenshtein distance of 1 from the target package, it is unlikely that a developer would purposely import any of the former packages in place of *buffer-xor*. Typos are likewise unlikely due to the significant distance between swapped character on most keyboard layouts. As elucidated in Section 3, we found edit distance to be a poor metric for typosquatting, and therefore we consciously avoid flagging those cases, which would result in an unmanageable number of warnings anyway.

Instead, to get a sense of how each of the signals were performing, we considered the number of packages in each repository which match a given signal. These results are shown in Table 3. Note that these figures contain no notion of dependencies and, are therefore, not transitive. Here we are interested in examining how aggressive each of the signals are. Interestingly, despite npm having about 6 times as many packages as PyPI, the number of npm packages which fit our definition of typosquatting is almost 10 times higher. This result points toward the conclusion that typosquatting is inherently a larger issue in npm.

4.4 SpellBound Overhead

The goal of our final analysis of SpellBound is to determine the temporal overhead it imposes on the package installation process. To quantify the performance of SpellBound, 1,000 npm packages were selected at random, weighted by popularity. Weighting the selections during this process is crucial, as it creates a sample that simulates the downloading patterns of actual repository users. Once selected, the contents of these packages were locally cached to remove any uncontrollable network-based effects on installation times. After being cached, installation times for each package were measured using npm's official package manager and a version modified to implement SpellBound. The official npm package manager had an average installation time of 2.604 seconds, while SpellBound resulted in an average installation time of 2.669 seconds, meaning SpellBound imposes an **average temporal overhead of about 2.5%**. We believe this result is reasonable and the slowdown incurred by SpellBound is effectively unnoticeable.

Batch mode performance: Batch mode (ref. Section 3.2) analyzes the entire package set in a single pass and is intended to be used by repository maintainers to discover yet unknown issues of typosquatting. In our experiments, we found that SpellBound can analyze the entire npm package set in **11 minutes**. This result suggests that SpellBound could be run frequently (e.g., once per day) allowing quick identification of unknown typosquatting cases.

5 Discussion

In this section, we discuss the broader implications of our findings, include some of the subtleties related to typosquatting, explore possible steps to mitigate typosquatting beyond SpellBound, and consider alternative ways to implement SpellBound.

5.1 Alternative SpellBound Deployments

As discussed in Section 3, our primary deployment of SpellBound is a modification to existing package manager frontend tools. Implementing our tool in this way allows typosquatting protection to be non-invasive and fit into existing workflows. Ultimately, we hope that our mechanism is incorporated into existing package management tools. However, we also implemented a standalone command-line tool that performs our transitive typosquatting protection checks without the cooperation of the frontend, thus allowing users to avail themselves of typosquatting protection even if such protection is not directly integrated in the package manager.

The goal of SpellBound is to decrease the chances that a user of a package manager will accidentally install an incorrect package due to typosquatting. However, it is beyond the scope of this work to model all of the ways in which a user might confuse their target package name. For example, confusion may stem from misremembering a name, or hearing it incorrectly. Similarly, the particular keyboard layout used by a developer influences the typos that that developer is likely to make when typing in the package name. Collectively, these differences may justify personalizing the typosquatting detection scheme.

SpellBound relies on the concept of popularity. It is possible to define alternative notions of popularity by changing the metric with which the popularity of a given package is quantified (e.g. using the number of dependent packages). Exploring these alternatives is future work. An additional implementation detail of our detection algorithm is that it considers potential victim packages and potential perpetrator packages to be disjoint sets partitioned by the popularity threshold. A natural extension would be to consider these sets to overlap, such that somewhat popular packages could be classified as typosquatting perpetrators or victims.

The evaluation results in Section 4 show that the perpetrator package detection algorithm developed as part of this work is unobtrusive, but detects real cases of typosquatting. Nevertheless, the modular design of SpellBound means that the alternative approaches outlined above could be dropped in to the tool with no changes to the workflow.

5.2 Server-Side Protection Mechanisms

Our technique successfully detected typosquatting that was active in popular package repositories for over a year, leading to effective remediation: developers updated their dependencies to their intended target package, and repository maintainers seized and deprecated the perpetrator package. Consequently, we feel that our approach could aid server-side security teams in scanning their entire repository to discovered latent typosquatting instances. As discussed in Section 3, repository maintainers can run SpellBound in batch mode to identify suspicious packages that have already been uploaded. We also consider some additional mechanisms that may help to combat the typosquatting problems.

Preemptive takedown: An aggressive extension to server-side batch mode operation of SpellBound is to invoke a typoquatting check at the time a new package is uploaded, effectively disallowing the existence of too-similar package names. This proactive approach is a natural extension to the case-insensitive and delimiter-based naming restrictions currently in place on npm and PyPI [4–6]. It further limits the potential of a perpetrator package from gaining traction and achieving legitimacy through the confusion of users. We note that an implicit assumption of our current approach is that popular packages cannot, by definition, perpetrate typosquatting attacks. Our definition means that if an illegitimate package gains enough traction to exceed the threshold, it can avoid triggering a warning on installation. Disallowing the perpetrator package from being uploaded obviates that issue.

Variant-insensitive package names: Much like disallowing too-similar package names, a repository could map all variations of a package name to the canonical version of the package. This approach means that the perpetrator would be unable to upload their package, since the system would consider the name to be taken by the target package. Furthermore, it would address the typo by suggesting the correct target. Some repositories already implement some limited form of this behavior. PyPI maps all punctuation to hyphens and handles all package installation requests in a case-insensitive manner [4]. We believe such changes warrant future research. A potential concern with allowing all variations of a name to map to the same package is that it crowds the set of possible names. We note that npm already incorporates a typo-safe mechanism to allow similar package names, called *scoped packages* [3]. The mechanism works by allowing package names to begin with an @ symbol, followed by a namespace portion (typically the package creator's username), followed by a forward slash, followed by the basename of the package. Versions of many popular packages deployed using TypeScript (a typed superset of JavaScript) are available under the @types/ namespace (e.g. @types/node for the TypeScript version of the node package). Scoped packages can be used to alleviate the concern that a repository's names may become too crowded for a new package to be given a descriptive name.

5.3 Defensive Typosquatting

One tactic currently used to prevent package typosquatting is to preemptively register confusable variants alongside the canonical package name, so that the variants cannot fall under the control of a typosquatter. We refer to this tactic as *defensive typosquatting*. In the absence of officially supported mechanisms for defensive typosquatting, a benign placeholder package will be registered under the variant name. We observed instances of defensive typosquatting in both npm and PyPI, where package creators (or 3rd party package developers) are free to create as many packages as they desire with varying behaviors for placeholder packages. The placeholder behaviors that we observed are as follows:

Transparent inclusion of target package functionality: One approach is to transparently provide the functionality of the target package to the user within the placeholder package. This approach can be accomplishing with varying degrees of sophistication. By leveraging the repository's dependence mechanism, a placeholder package creators can effectively implement a passthrough to the intended target package. By making the legitimate package a dependency of the placeholder, the correct package is installed despite the request being incorrect. When included in a project, the defensive typosquatting package can simply import the legitimate package's functionality. We observed this behavior in practice in the npm package *buynan*, which (defensively) typosquats the legitimate package *bunyan*. The *buynan* package simply imports *bunyan* upon its inclusion. One limitation of this defense is that it is indiscernible from a case of a malicious Trojan package; at any point a 3rd-party owner of a placeholder could change the redirect to a malicious payload. Furthermore, a less sophisticated method for transparently including target package functionality is to clone the code of the target. However, if the placeholder fails to stay up-to-date with the package it defends, it can actually expose the user to latent vulnerabilities, effectively becoming a stale package typosquatting perpetrator. This was the particular situation *loadsh* was in when discovered.

User alerts: One possible option is to make the placeholder issue an informative alert with directions to change to the legitimate package. For example, the placeholder could print a message during at install time or runtime to inform the users of their mistake. This approach has been extensively used within the PyPI repository [8, 18]. In this case, placeholder packages utilize the install hook mechanism of PyPI to issue a message at install time that directs users to the packages they likely had in mind.

Package Deprecation: One mechanism used in practice to alert users that they should change packages is the deprecation mechanism. This mechanism allows a package maintainer to indicate that it should no longer be used. When a deprecated package is installed, the user is presented with an alert. Deprecation is used in practice when a stale package typosquatting perpetrator is discovered, since it does not break dependant code but still admonishes victims to update their dependencies. One limitation of this technique is that deprecation is a mechanism that is used for a variety of purposes. It is unclear whether the deprecation mechanism is sufficient to alert users of the type of error that they have incurred.

Defensive typosquatting will continue to have a place as a stopgap mechanism to protect against package name confusion, even in the presence of automated defenses like SpellBound, since the confusion may occur due to the specific context of the package name. Nevertheless, tools like SpellBound can ease the burden of placing placeholder packages.

6 Related Work

Typosquatting and Defenses: Tschacher's Bachelor thesis [38] demonstrates high success rates of a controlled typosquatting attack, proving the importance of devising countermeasures. It also briefly outlines defenses based on forbidding names similar to those of popular packages, but does not implement or evaluate them, and does not consider involving developers in the decision process.

The creators of npm and PyPI have taken basic countermeasures to combat typosquatting. Rules governing package names for these platforms have become increasingly strict in an attempt to mitigate the problem. Both platforms have incorporated restrictions on capitalization and punctuation-based differences [4, 5]. User-led defense campaigns exist that aim to "park" potential typosquatting names before they can be used in a malicious manner [7, 8].

Domain Name Typosquatting: Domain name typosquatting has long been a popular attack vector, allowing cybercriminals to hijack web communications [35] and potentially emails [36]. Such hijacking is typically operated by registering a domain name similar to a popular one. It is used for financial gain, by serving ads, pushing drive-by downloads, or orchestrating phishing attacks. In particularly serious cases, regulations such as the US ACPA allow ICANN to seize typosquatted domains to prevent confusion [1]. Such legal framework does not exist for package typosquatting, and indeed this approach may be difficult to apply due to the fast-evolving nature of software ecosystems. Furthermore, not all instances of package name typosquatting are the result of explicit attacks.

Software Ecosystem Security: Most past efforts focused on vulnerabilities of package managers themselves [16, 21], or potential attack strategies enacted by malicious packages [31]. Both goals are orthogonal to ours, and none of these works reviewed actual incidents or performed measurements on the extent of the problem.

Other works more specifically analyze security risks arising from the presence of malicious packages in highly inter-

connected software ecosystems [28, 44]. [44] also identifies typosquatting as one of multiple possible avenues for attack, but it provides no in-depth analysis of the phenomenon, nor describes solutions.

General Characterization of Software Ecosystems: Literature presents many other analyses of software ecosystems. While these works present useful information for understanding these complex objects, they do not focus on typosquatting or other potential security-related issues. Examples include [26, 33, 42].

Mobile Ecosystems: A related line of work is on the study of mobile application markets such as the Google Play store [22, 23, 39, 41]. These works are primarily concerned with applications used by consumers, rather than application components (packages) that are specific to the language ecosystem and used by developers. As such, characterization of app markets (and defenses proposed against malicious applications) are largely orthogonal to our work. The closest work is in the detection of *cloned* applications, whereby a lesser-known or actively malicious developer will re-package and re-publish a better-known app. Detecting application clones has typically been done via code similarity metrics [27] or behavior [24]. In contrast, our approach is based entirely on the package metadata and an analysis of the properties of the package repository.

Supply Chain Vulnerabilities: Others have looked at the related problem of *supply chain vulnerabilities*, i.e., vulnerabilities in the open-source applications on which a software package depends [20, 30, 32, 37, 43]. These works typically discuss identification or impact of potential upstream vulnerabilities. While an attacker could attempt to introduce such a vulnerability via typosquatting, analyzing this possibility is outside the scope of our work.

7 Conclusion

Package managers vastly improve the software development workflow. They can quickly download and install third-party packages, along with any dependencies, to import constructive functionality into a project. Packages are typically requested explicitly by name and currently, there exists no safety net for developers during the package installation process. Typosquatting attacks target those who make a spelling mistake and their effects can be severe. These attacks are far from novel due to their extensive history of targeting domain names. Although, the focus of typosquatting attacks has recently grown to include package repositories. Despite hundreds of past attacks, practical defenses against typosquatting in package repositories such as npm and PyPI have received little attention.

In this paper, we have shown that a defense against these attacks is both practical and efficient. By comparing the name in the requested package's dependency tree to a list of probable targets, our proposed solution can protect developers from typosquatting attacks. With an average overhead of 2.5%, a warning-to-install ratio of 0.5%, and third-party confirmation of flagged packages, our solution imposes a negligible burden while protecting package creators and end users alike.

References

[1] Senate Report 106-140 - THE ANTICYBERSQUATTING CONSUMER PROTECTION ACT, August 1999.

[2] The npm blog - numeric precision matters: how npm download counts work, July 2014.

[3] npm-scope | npm documentation, August 2015.

[4] Pep 503 – simple repository api, September 2015.

[5] New package moniker rules, December 2017.

[6] The npm blog - 'crossenv' malware on the npm registry, August 2017.

[7] pypi-parker, October 2017.

[8] Pypi user - wbengtson, January 2018.

[9] Malicious package report: browserift - snyk.io, July 2019.

[10] Malicious package report: comander - snyk.io, October 2019.

[11] Malicious package report: destroyer-of-worlds - snyk.io, May 2019.

[12] Malicious package report: device-mqtt - snyk.io, August 2019.

[13] npm security advisory: babel-laoder, November 2019.

[14] npm security advisory: sj-tw-sec, November 2019.

[15] Prototype pollution in lodash | snyk, July 2019.

[16] Anish Athalye, Rumen Hristov, Tran Nguyen, and Qui Nguyen. Package Manager Security. Technical report.

[17] Stefan Axelsson. The base-rate fallacy and its implications for the difficulty of intrusion detection. In *Proceedings of the 6th ACM conference on Computer and communications security - CCS '99*, pages 1–7. ACM Press, 1999.

[18] Ethan Bommarito and Michael Bommarito. An empirical analysis of the python package index (pypi). *arXiv preprint arXiv:1907.11073*, 2019.

[19] Rainer Böhme and Jens Grossklags. The security cost of cheap user interaction. In *Proceedings of the 2011 workshop on New security paradigms workshop - NSPW '11*. ACM Press, 2011.